

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_ARGS 64
#define BUF_SIZE 1024

// Global state tracking for previous directory (OLDPWD)
char prev_dir[PATH_MAX] = "";

// Dispatch Table function pointer signature
typedef int (*BuiltinHandler)(char **args);

typedef struct {
    const char *name;
    BuiltinHandler handler;
} BuiltinCommand;

// Forward declarations
int builtin_cd(char **args);
int builtin_pwd(char **args);
int builtin_help(char **args);

// Dispatch Table for Built-in Commands
BuiltinCommand dispatch_table[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"help", builtin_help},
    {NULL, NULL}
};

// 1. CD BUILT-IN HANDLER
int builtin_cd(char **args) {
    char current_dir[PATH_MAX];
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
```

```

        perror("getcwd failed");
        return 1;
    }

    const char *target_dir = args[1];

    // Case A: No arguments passed (cd) -> default to $HOME
    if (target_dir == NULL || strcmp(target_dir, "~") == 0) {
        target_dir = getenv("HOME");
        if (target_dir == NULL) {
            printf("cd: HOME variable not set\n");
            return 1;
        }
    }

    // Case B: Handle 'cd -' -> toggle to previous directory
    else if (strcmp(target_dir, "-") == 0) {
        if (strlen(prev_dir) == 0) {
            printf("cd: OLDPWD not set\n");
            return 1;
        }
        target_dir = prev_dir;
        printf("%s\n", target_dir); // Print target when using cd -
    }

    // Attempt directory change & handle errors (e.g., path doesn't exist)
    if (chdir(target_dir) != 0) {
        perror("cd failed");
        return 1;
    }

    // Update state: store previous directory and update PWD / OLDPWD
    strncpy(prev_dir, current_dir, sizeof(prev_dir));
    setenv("OLDPWD", current_dir, 1);

    if (getcwd(current_dir, sizeof(current_dir)) != NULL) {
        setenv("PWD", current_dir, 1);
    }

```

```

    return 0;
}

// 2. PWD BUILT-IN HANDLER
int builtin_pwd(char **args) {
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd failed");
        return 1;
    }
    return 0;
}

// Help menu
int builtin_help(char **args) {
    printf("--- Directory Navigation Commands ---\n");
    printf("1. cd <path> : Change directory\n");
    printf("2. cd -      : Change to previous directory\n");
    printf("3. cd        : Change to home directory\n");
    printf("4. pwd       : Display current working directory\n");
    printf("5. help      : Display menu\n");
    printf("6. exit      : Terminate shell\n");
    return 0;
}

// Dispatcher logic
int execute_builtin(char **args) {
    if (args[0] == NULL) return 0;

    for (int i = 0; dispatch_table[i].name != NULL; i++) {
        if (strcmp(args[0], dispatch_table[i].name) == 0) {
            return dispatch_table[i].handler(args);
        }
    }
    return -1; // Not a built-in
}

```

```

}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        char cwd_prompt[PATH_MAX];
        if (getcwd(cwd_prompt, sizeof(cwd_prompt)) == NULL) {
            strcpy(cwd_prompt, "shell");
        }

        printf("\n[%s] > ", cwd_prompt);
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0;

        if (strcmp(input_buf, "exit") == 0) break;

        // Tokenize input
        char *args[MAX_ARGS];
        int arg_count = 0;
        char *token = strtok(input_buf, " ");

        while (token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " ");
        }
        args[arg_count] = NULL;

        // Execute via dispatch table
        if (arg_count > 0) {
            int result = execute_builtin(args);
            if (result == -1) {
                printf("Error: '%s' is an invalid or non-builtin command.\n", args[0]);
            }
        }
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano cd_dispatch.c
abhis@MIKHAIL-37:~$ gcc -g -Wall cd_dispatch.c -o cd_dispatch
abhis@MIKHAIL-37:~$ ./cd_dispatch

[/home/abhis] > pwd
/home/abhis

[/home/abhis] > cd /tmp

[/tmp] > cd -
/home/abhis

[/home/abhis] > cd

[/home/abhis] > cd /nonexistent_folder_123
cd failed: No such file or directory

[/home/abhis] > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./cd_dispatch
==1337== Memcheck, a memory error detector
==1337== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1337== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1337== Command: ./cd_dispatch
==1337==

[/home/abhis] > exit
==1337==
==1337== HEAP SUMMARY:
==1337==    in use at exit: 0 bytes in 0 blocks
==1337==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1337==
==1337== All heap blocks were freed -- no leaks are possible
==1337==
==1337== For lists of detected and suppressed errors, rerun with: -s
==1337== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```